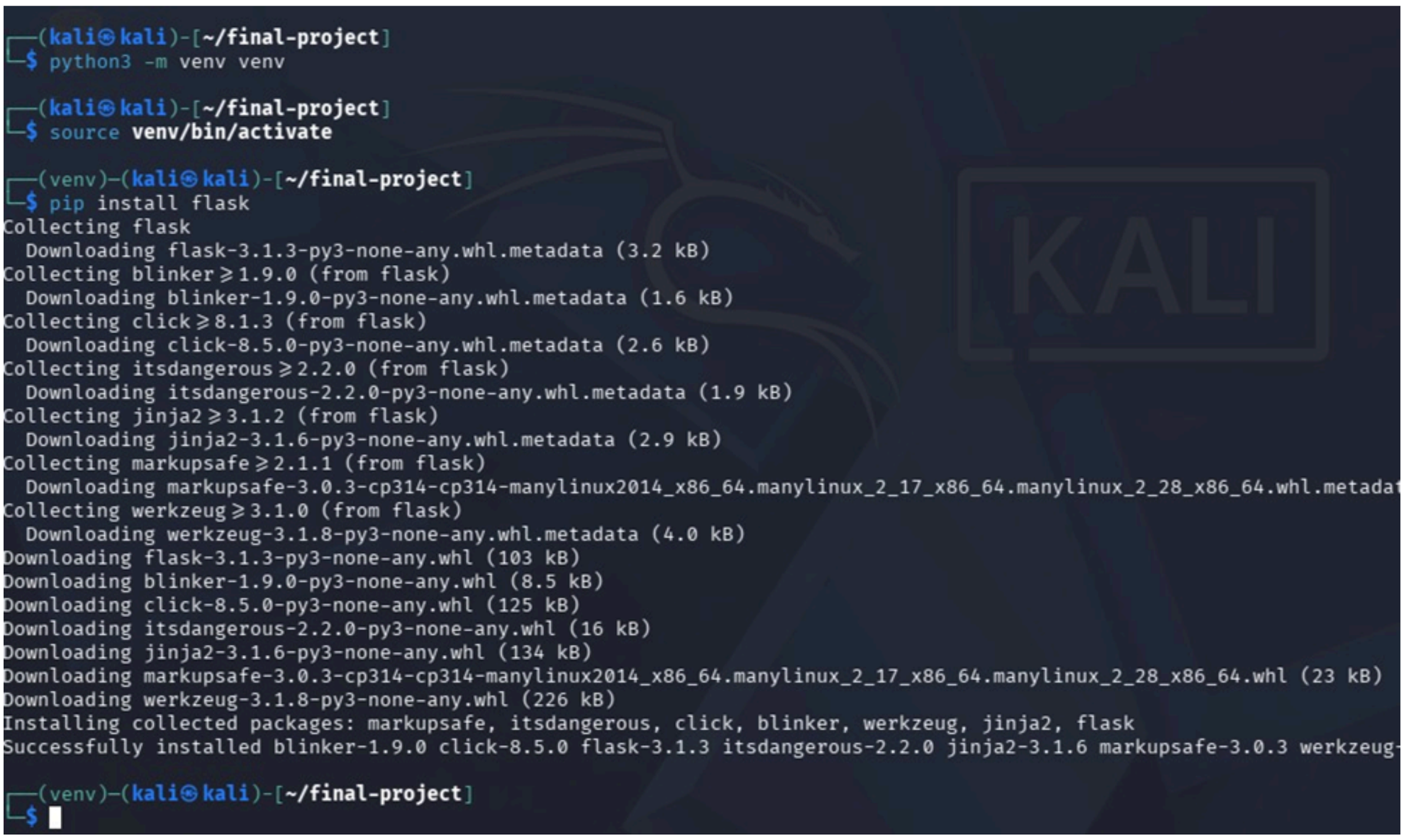
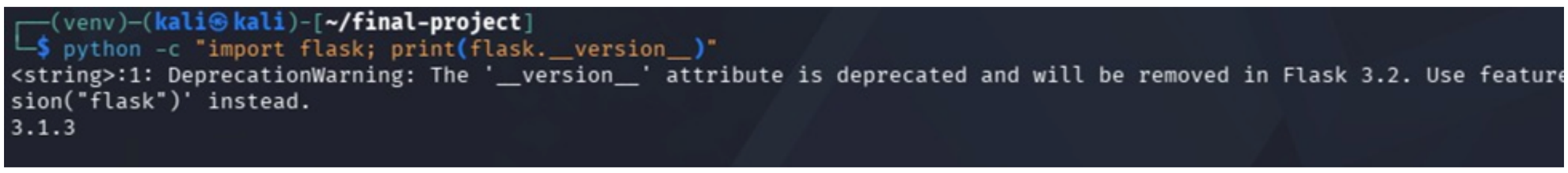
Web Application Security Assessment and Secure Remediation

application overview and technology stack

the vulnerable web application environment was prepared using flask as the main development framework. a dedicated project directory was created under `/home/kali/final-project` and a python virtual environment was configured to isolate the project dependencies. flask was then installed successfully and verified using python. the application was developed as a controlled vulnerable web application for the penetration testing phase.





```
┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ python -c "import flask; print(flask.__version__)"
<string>:1: DeprecationWarning: The '__version__' attribute is deprecated and will be removed in Flask 3.2. Use feature
sion("flask")' instead.
3.1.3

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ pwd
/home/kali/final-project

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ python --version

Python 3.14.6

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ pip --version
pip 26.1.2 from /home/kali/final-project/venv/lib/python3.14/site-packages/pip (python 3.14)

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ python -c "import flask; print(flask.__version__)"
<string>:1: DeprecationWarning: The '__version__' attribute is deprecated and will be removed in Flask 3.2. Use feature
sion("flask")' instead.
3.1.3

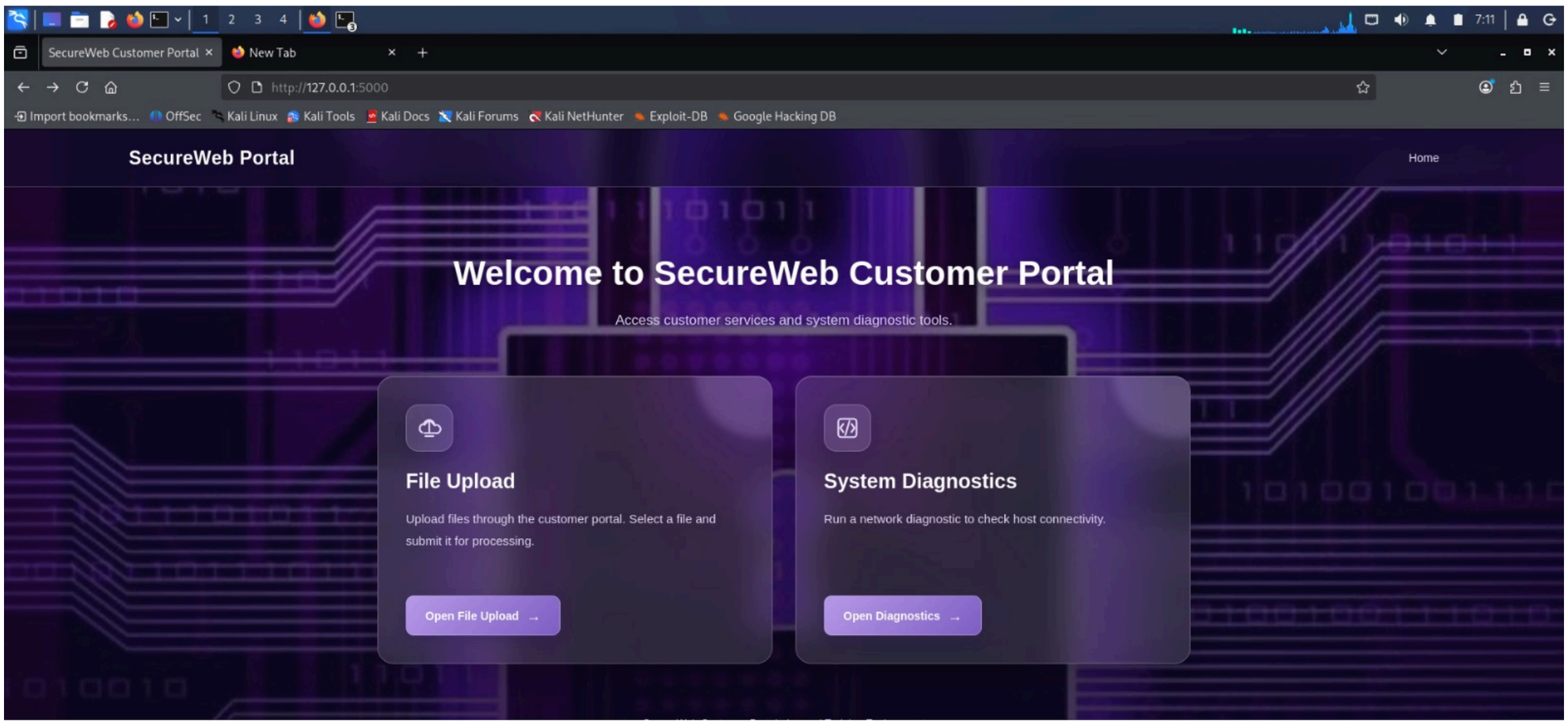
┌──(venv)─(kali㉿kali)-[~/final-project]
└─$
```

2.3.1 application overview and technology stack:

the vulnerable web application was developed from scratch using flask as the main development framework. a dedicated project directory was created under `/home/kali/final-project` and a python virtual environment was configured to isolate the project dependencies. flask was installed successfully and verified using python. the application was then configured to run on port 5000 and accessed through the web browser. the application provides a customer portal with two main functionalities: file upload and system diagnostics.

application main page:

the main page was created using html and provides access to the customer portal, file upload functionality, and system diagnostics functionality. the page was designed as the main entry point to the application and provides navigation to the available application features.




```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SecureWeb Customer Portal</title>

  <style>
    *{
      box-sizing: border-box;
    }

    body{
      margin: 0;
      min-height: 100vh;
      font-family: Arial, sans-serif;
      color: white;

      background:
        linear-gradient(
          rgba(24, 15, 58, 0.55),
          rgba(15, 10, 38, 0.78)
        ),
        url("/static/sky-background.png");

      background-size: cover;
      background-position: center;
      background-repeat: no-repeat;
      background-attachment: fixed;
```

```
}

.navbar{
    background: rgba(20, 12, 48, 0.58);
    backdrop-filter: blur(14px);
    -webkit-backdrop-filter: blur(14px);

    color: white;
    padding: 22px 8%;

    display: flex;
    justify-content: space-between;
    align-items: center;

    border-bottom: 1px solid rgba(255, 255, 255, 0.18);
}

.brand{
    font-size: 23px;
    font-weight: bold;
    letter-spacing: 0.5px;
}

.nav-link{
    color: #eee7ff;
    text-decoration: none;
    font-size: 14px;
    transition: color 0.3s ease;
}

.nav-link:hover{
    color: white;
}

.container{
    max-width: 1050px;
    margin: 0 auto;
    padding: 80px 25px 35px;
}

.hero{
    text-align: center;
    margin-bottom: 55px;
}
```

```
.hero h1 {
    margin: 0 0 18px;

    font-size: 42px;
    line-height: 1.2;
    font-weight: bold;

    color: #ffffff;
    text-shadow: 0 4px 20px rgba(0, 0, 0, 0.4);
}

.hero p {
    margin: 0;
    color: #e1d8f5;
    font-size: 17px;
    line-height: 1.7;
}

.services {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 28px;
}

.card {
    position: relative;
    overflow: hidden;

    background: rgba(255, 255, 255, 0.13);
    backdrop-filter: blur(18px);
    -webkit-backdrop-filter: blur(18px);

    border: 1px solid rgba(255, 255, 255, 0.22);
    border-radius: 20px;

    padding: 34px;

    box-shadow:
        0 15px 45px rgba(0, 0, 0, 0.28),
        inset 0 1px 0 rgba(255, 255, 255, 0.12);

    transition:
        transform 0.3s ease,
        background 0.3s ease,
        box-shadow 0.3s ease;
```

```
}

.card::before{
  content: "";
  position: absolute;

  width: 160px;
  height: 160px;

  top: -85px;
  right: -65px;

  background: rgba(216, 190, 255, 0.2);
  border-radius: 50%;
  filter: blur(5px);
}

.card:hover{
  transform: translateY(-8px);
  background: rgba(255, 255, 255, 0.18);

  box-shadow:
    0 20px 55px rgba(0, 0, 0, 0.38),
    inset 0 1px 0 rgba(255, 255, 255, 0.2);
}

.card-icon{
  position: relative;

  width: 58px;
  height: 58px;

  display: flex;
  align-items: center;
  justify-content: center;

  margin-bottom: 22px;

  border-radius: 16px;

  background: rgba(205, 181, 246, 0.2);
  border: 1px solid rgba(255, 255, 255, 0.22);

  box-shadow:
    0 8px 20px rgba(30, 15, 65, 0.25);
```

```
}

.card-icon svg{
    width: 30px;
    height: 30px;

    fill: none;
    stroke: #eadfff;
    stroke-width: 1.7;
    stroke-linecap: round;
    stroke-linejoin: round;
}

.card h2{
    position: relative;

    margin: 0 0 18px;
    color: #ffffff;
    font-size: 25px;
}

.card p{
    position: relative;

    color: #e5dcf8;
    line-height: 1.8;
    font-size: 15px;
    min-height: 75px;
}

.button{
    position: relative;

display: inline-flex;
align-items: center;
justify-content: center;

margin-top: 18px;

background: linear-gradient(
    135deg,
    #b99bea,
    #8060c5
);
```

```
    color: white;
    text-decoration: none;

    padding: 13px 23px;
    border-radius: 10px;

    font-weight: bold;
    font-size: 14px;

    border: 1px solid rgba(255, 255, 255, 0.25);

    box-shadow:
      0 8px 20px rgba(45, 25, 90, 0.4);

    transition:
      transform 0.3s ease,
      background 0.3s ease,
      box-shadow 0.3s ease;
}

.button:hover {
    background: linear-gradient(
      135deg,
      #cdb5f6,
      #9474d5
    );

    transform: translateY(-2px);

    box-shadow:
      0 12px 25px rgba(45, 25, 90, 0.55);
}

.arrow {
    display: inline-block;
    margin-left: 9px;
    font-size: 18px;

    transition: transform 0.3s ease;
}

.button:hover .arrow {
    transform: translateX(4px);
}
```

```
    .footer{
      text-align: center;
      margin-top: 65px;

      color: #c2b3df;
      font-size: 13px;
    }

    @media (max-width: 650px){
      .navbar{
        padding: 20px;
      }

      .container{
        padding: 55px 20px 30px;
      }

      .hero h1 {
        font-size: 30px;
      }

      .hero p{
        font-size: 15px;
      }

      .services{
        grid-template-columns: 1fr;
      }

      .card {
        padding: 28px;
      }
    }
  </style>
</head>

<body>

  <nav class="navbar">
    <div class="brand">SecureWeb Portal</div>
    <a class="nav-link" href="/">Home</a>
  </nav>

  <main class="container">
```

```
<section class="hero">
  <h1>Welcome to SecureWeb Customer Portal</h1>
  <p>
    Access customer services and system diagnostic tools.
  </p>
</section>

<section class="services">

  <!-- File Upload Card -->
  <div class="card">

    <div class="card-icon">
      <svg viewBox="0 0 24 24" aria-hidden="true">
        <path d="M12 16V4"></path>
        <path d="M7 9l5-5 5 5"></path>
        <path d="M5 16.5A4.5 4.5 0 0 0 6.5 18a5.5 5.5 0 0 1 10.7 1.5A4 4 0 1 1 18 17H7"></path>
        <path d="M8 20h8"></path>
      </svg>
    </div>

    <h2>File Upload</h2>

    <p>
      Upload files through the customer portal.
      Select a file and submit it for processing.
    </p>

    <a class="button" href="/upload">
      Open File Upload
      <span class="arrow">→</span>
    </a>

  </div>

  <!-- System Diagnostics Card -->
  <div class="card">

    <div class="card-icon">
      <svg viewBox="0 0 24 24" aria-hidden="true">
        <rect x="3" y="3" width="18" height="18" rx="3"></rect>
        <path d="M8 8l-3 4 3 4"></path>
        <path d="M16 8l3 4-3 4"></path>
        <path d="M14 5l-4 14"></path>
      </svg>
```

```
      </div>

      <h2>System Diagnostics</h2>

      <p>
        Run a network diagnostic to check
        host connectivity.
      </p>

      <a class="button" href="/command">
        Open Diagnostics
        <span class="arrow">→</span>
      </a>

    </div>

  </section>

  <div class="footer">
    SecureWeb Customer Portal · Internal Training Environment
  </div>

 </main>

</body>
</html>
```

```
┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ cd ~/final-project

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ mkdir -p templates uploads

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ pwd
/home/kali/final-project

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ ls -la
total 24
drwxrwxr-x  5 kali kali 4096 Sep 11 00:12 .
drwx------ 23 kali kali 4096 Sep 10 23:48 ..
-rw-rw-r--  1 kali kali  778 Sep 11 00:04 app.py
drwxrwxr-x  2 kali kali 4096 Sep 11 00:12 templates
drwxrwxr-x  2 kali kali 4096 Sep 11 00:12 uploads
drwxrwxr-x  5 kali kali 4096 Sep 10 23:53 venv

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ nano templates/upload.html

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ ls -l templates/
total 4
-rw-rw-r-- 1 kali kali 331 Sep 11 00:13 upload.html

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$
```

```
┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ nano templates/upload.html

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ ls -l templates/
total 4
-rw-rw-r-- 1 kali kali 331 Sep 11 00:13 upload.html

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ nano templates/index.html

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ ls -l templates/
total 8
-rw-rw-r-- 1 kali kali 226 Sep 11 00:16 index.html
-rw-rw-r-- 1 kali kali 331 Sep 11 00:13 upload.html

┌──(venv)─(kali㉿kali)-[~/final-project]
└─$
```

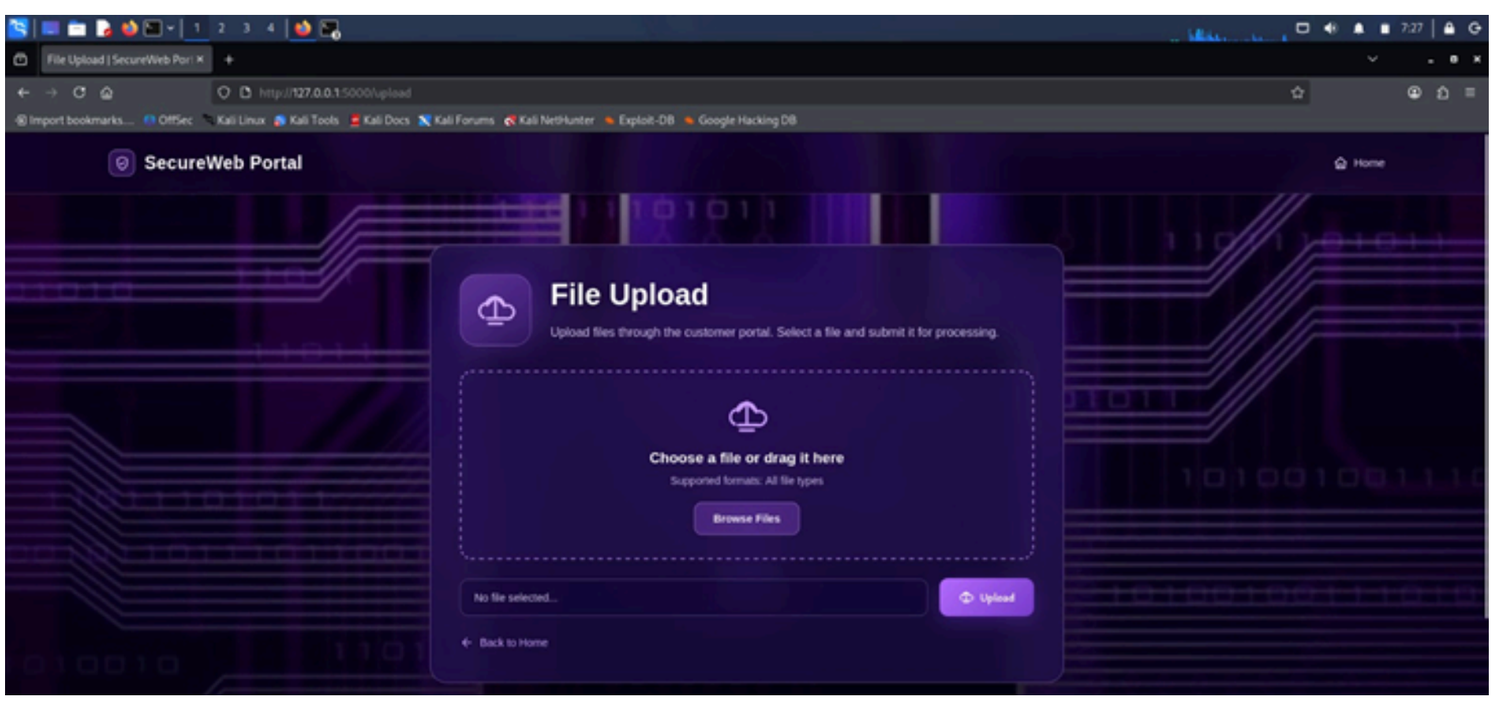
```
┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.50.128:5000
Press CTRL+C to quit
```

2.3.2 vulnerability 1 file upload:

a file upload functionality was implemented in the vulnerable web application using flask. the application provides an upload page where users can select and submit files through the `/upload` endpoint. the uploaded files are saved directly into the `uploads` directory without restricting file types or validating the uploaded filename. this implementation was intentionally configured without security controls in order to represent the unrestricted file upload vulnerability required for the penetration testing phase.

upload page implementation:

the upload page was created using html and provides an interface where users can select a file and submit it to the application. the form uses the multipart form encoding required for transferring files through an http request. the page also includes the upload button and navigation back to the main customer portal.



the upload interface contains a file selection field and an upload button. the selected file is submitted to the flask application through the `/upload` endpoint using a post request.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>File Upload | SecureWeb Portal</title>
```

```
<style>
  *{
    box-sizing: border-box;
  }

  body{
    margin: 0;
    min-height: 100vh;
    font-family: Arial, sans-serif;
    color: white;

    background:
      linear-gradient(
        rgba(20, 5, 45, 0.55),
        rgba(8,5, 25, 0.82)
      ),
      url("/static/upload-background.png");

    background-size: cover;
    background-position: center;
    background-repeat: no-repeat;
    background-attachment: fixed;

    display: flex;
    flex-direction: column;
}

 .navbar{
    background: rgba(18, 5, 42, 0.72);
    backdrop-filter: blur(14px);
    -webkit-backdrop-filter: blur(14px);

    padding: 22px 7%;

    display: flex;
    justify-content: space-between;
    align-items: center;

    border-bottom: 1px solid rgba(255, 255, 255, 0.18);
}

 .brand{
    display: flex;
    align-items: center;
```

```
    gap: 12px;

    font-size: 23px;
    font-weight: bold;
    letter-spacing: 0.5px;
}

.brand-icon{
    width: 34px;
    height: 34px;

    display: flex;
    align-items: center;
    justify-content: center;

    border: 1px solid rgba(210, 160, 255, 0.6);
    border-radius: 10px;

    background: rgba(166,88, 255, 0.2);
}

.brand-icon svg{
    width: 21px;
    height: 21px;

    fill: none;
    stroke: #d9b7ff;
    stroke-width: 1.8;
    stroke-linecap: round;
    stroke-linejoin: round;
}

.nav-link{
    display: flex;
    align-items: center;
    gap: 8px;

    color: #eee2ff;
    text-decoration: none;
    font-size: 15px;

    transition: color 0.3s ease;
}

.nav-link:hover{
```

```
    color: white;
}

.nav-link svg {
    width: 19px;
    height: 19px;

    fill: none;
    stroke: currentColor;
    stroke-width: 1.8;
    stroke-linecap: round;
    stroke-linejoin: round;
}

.main-content{
    flex: 1;

    display: flex;
    align-items: center;
    justify-content: center;

    padding: 65px 20px;
}

.upload-card{
    width: 100%;
    max-width: 820px;

    padding: 38px;

    background: rgba(31, 17, 75, 0.68);
    backdrop-filter: blur(20px);
    -webkit-backdrop-filter: blur(20px);

    border: 1px solid rgba(220, 184, 255, 0.35);
    border-radius: 22px;

    box-shadow:
      0 20px 65px rgba(0, 0, 0, 0.42),
      inset 0 1px 0 rgba(255, 255, 255, 0.12);
}

.heading-area {
    display: flex;
    align-items: center;
```

```
    gap: 25px;

    margin-bottom: 32px;
  }

  .main-icon {
    width: 92px;
    height: 92px;

    flex-shrink: 0;

    display: flex;
    align-items: center;
    justify-content: center;

    border-radius: 22px;

    background:
      linear-gradient(
        145deg,
        rgba(183, 111, 255, 0.4),
        rgba(90, 36, 170, 0.25)
      );

    border: 1px solid rgba(220, 184, 255, 0.35);

    box-shadow:
      0 0 30px rgba(171, 82, 255, 0.25);
  }

  .main-icon svg {
    width: 52px;
    height: 52px;

    fill: none;
    stroke: #d8b2ff;
    stroke-width: 1.5;
    stroke-linecap: round;
    stroke-linejoin: round;
  }

  .heading-text h1 {
    margin: 0 0 12px;

    font-size: 38px;
```

```
    color: white;

    text-shadow:
      0 4px 18px rgba(0,0,0,0.35);
}

.heading-text p{
    margin: 0;

    color: #d8c8f2;
    font-size: 16px;
    line-height: 1.7;
}

.upload-zone{
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;

    min-height: 220px;

    padding: 30px;

    border: 2px dashed rgba(202, 153, 255, 0.75);
    border-radius: 16px;

    background: rgba(42, 20, 91,0.38);

    text-align: center;

    transition:
      background 0.3s ease,
      border-color 0.3s ease;
}

.upload-zone:hover{
    background: rgba(73, 35, 135, 0.42);
    border-color: #d4aaff;
}

.zone-icon{
    width: 54px;
    height: 54px;
```

```
    margin-bottom: 15px;
}

.zone-icon svg{
    width: 54px;
    height: 54px;

    fill: none;
    stroke: #cda0ff;
    stroke-width: 1.5;
    stroke-linecap: round;
    stroke-linejoin: round;
}

.upload-zone h2{
    margin: 0 0 10px;

    font-size: 19px;
    color: white;
}

.upload-zone p{
    margin: 0 0 20px;

    color: #cbbbe7;
    font-size: 14px;
}

.choose-button{
    display: inline-block;

    padding: 12px 24px;

    border-radius: 10px;

    background: rgba(161, 91, 237, 0.3);
    border: 1px solid rgba(214, 170, 255, 0.5);

    color: #f5edff;
    font-weight: bold;
    font-size: 14px;

    cursor: pointer;

    transition:
```

```
        background 0.3s ease,
        transform 0.3s ease;
    }

    .choose-button:hover{
        background: rgba(175, 105, 255, 0.48);
        transform: translateY(-2px);
    }

    #file{
        display: none;
    }

    .file-row{
        display: flex;
        align-items: center;
        gap: 15px;

        margin-top: 24px;
    }

    .file-name{
        flex: 1;
        min-width: 0;

        padding: 15px 18px;

        border: 1px solid rgba(207, 173, 255, 0.3);
        border-radius: 11px;

        background: rgba(20, 8, 55, 0.42);

        color: #d8c9ef;
        font-size: 14px;

        overflow: hidden;
        text-overflow: ellipsis;
        white-space: nowrap;
    }

    .upload-button{
        display: inline-flex;
        align-items: center;
        justify-content: center;
        gap: 9px;
```

```
    padding: 15px 25px;

    border: none;
    border-radius: 11px;

    background: linear-gradient(
      135deg,
      #b46cff,
      #783dcc
    );

    color: white;
    font-size: 14px;
    font-weight: bold;

    cursor: pointer;

    box-shadow:
      0 8px 22px rgba(104, 43, 185, 0.4);

    transition:
      transform 0.3s ease,
      box-shadow 0.3s ease;
  }

  .upload-button:hover{
    transform: translateY(-2px);

    box-shadow:
      0 12px 28px rgba(104, 43, 185, 0.58);
  }

  .upload-button svg{
    width: 19px;
    height: 19px;

    fill: none;
    stroke: currentColor;
    stroke-width: 1.8;
    stroke-linecap: round;
    stroke-linejoin: round;
  }

  .back-link{
```

```
    display: inline-flex;
    align-items: center;
    gap: 8px;

    margin-top: 25px;

    color: #cda4ff;
    text-decoration: none;
    font-size: 14px;

    transition: color 0.3s ease;
}

.back-link:hover{
    color: white;
}

.back-link svg{
    width: 18px;
    height: 18px;

    fill: none;
    stroke: currentColor;
    stroke-width: 1.8;
    stroke-linecap: round;
    stroke-linejoin: round;
}

.footer{
    padding: 20px;

    text-align: center;

    color: #a99ac7;
    font-size: 13px;

    border-top: 1px solid rgba(255, 255, 255, 0.12);
    background: rgba(10, 4, 28, 0.45);
}

@media (max-width: 650px){
    .navbar{
        padding: 20px;
    }
```

```css
.brand{
  font-size: 18px;
}

.main-content{
  padding: 35px 15px;
}

.upload-card{
  padding: 25px 20px;
}

.heading-area{
  align-items: flex-start;
  gap: 16px;
}

.main-icon{
  width: 65px;
  height: 65px;
  border-radius: 17px;
}

.main-icon svg{
  width: 38px;
  height: 38px;
}

.heading-text h1{
  font-size: 28px;
}

.heading-text p{
  font-size: 14px;
}

.file-row{
  flex-direction: column;
  align-items: stretch;
}

.upload-button{
  width: 100%;
}
}
```

```
  </style>
</head>

<body>

  <nav class="navbar">

    <div class="brand">
      <div class="brand-icon">
        <svg viewBox="0 0 24 24" aria-hidden="true">
          <path d="M12 3l7 3v5c0 4.5-3 8.2-7 10-4-1.8-7-5.5-7-10V6l7-3z"></path>
          <path d="M9 12l2 2 4-4"></path>
        </svg>
      </div>

      <span>SecureWeb Portal</span>
    </div>

    <a class="nav-link" href="/">
      <svg viewBox="0 0 24 24" aria-hidden="true">
        <path d="M3 11l9-8 9 8"></path>
        <path d="M5 10v10h14V10"></path>
        <path d="M9 20v-6h6v6"></path>
      </svg>

      Home
    </a>

</nav>

<main class="main-content">

  <section class="upload-card">

    <div class="heading-area">

      <div class="main-icon">
        <svg viewBox="0 0 24 24" aria-hidden="true">
          <path d="M12 16V4"></path>
          <path d="M7 9l5-5 5 5"></path>
          <path d="M5 16.5A4.5 4.5 0 0 0 6.5 8a5.5 5.5 0 0 1 10.7 1.5A4 4 0 1 1 18 17H7"></path>
          <path d="M8 20h8"></path>
        </svg>
      </div>
```

```
      <div class="heading-text">
        <h1>File Upload</h1>

        <p>
          Upload files through the customer portal.
          Select a file and submit it for processing.
        </p>
      </div>

    </div>

    <form method="POST" enctype="multipart/form-data">

      <div class="upload-zone">

        <div class="zone-icon">
          <svg viewBox="0 0 24 24" aria-hidden="true">
            <path d="M12 16V4"></path>
            <path d="M7 9l5-5 5 5"></path>
            <path d="M5 16.5A4.5 4.5 0 0 1 6.58 8a5.5 5.5 0 0 1 10.71 1.5A4 4 0 1 1 18 17H7"></path>
            <path d="M8 20h8"></path>
          </svg>
        </div>

        <h2>Choose a file or drag it here</h2>

        <p>
          Supported formats: All file types
        </p>

        <label class="choose-button" for="file">
          Browse Files
        </label>

        <input
          type="file"
          id="file"
          name="file"
        >

      </div>

      <div class="file-row">

        <div class="file-name" id="file-name">
```

```
        No file selected...
      </div>

      <button class="upload-button" type="submit">

        <svg viewBox="0 0 24 24" aria-hidden="true">
          <path d="M12 16V4"></path>
          <path d="M7 9l5-5 5 5"></path>
          <path d="M5 16.5A4.5 4.5 0 0 1 6.58 8a5.5 5.5 0 0 1 10.71.5A4 4 0 1 1 18 17H7"></path>
          <path d="M8 20h8"></path>
        </svg>

        Upload

      </button>

    </div>

  </form>

  <a class="back-link" href="/">
    <svg viewBox="0 0 24 24" aria-hidden="true">
      <path d="M19 12H5"></path>
      <path d="M12 19l-7-7 7-7"></path>
    </svg>

    Back to Home
  </a>

</section>

</main>

<footer class="footer">
  SecureWeb Customer Portal · Internal Training Environment
</footer>

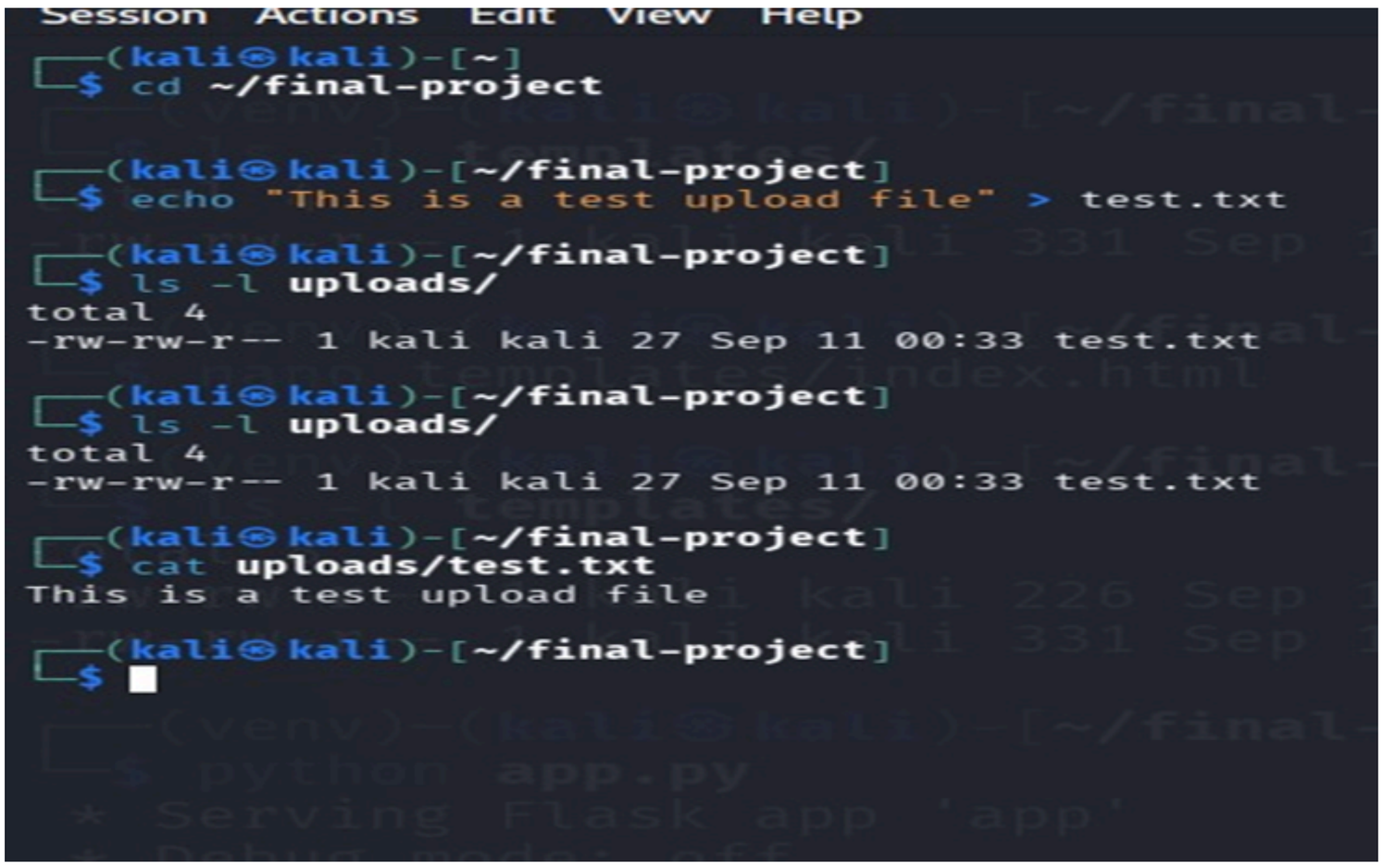
<script>
  const fileInput = document.getElementById("file");
  const fileName = document.getElementById("file-name");

  fileInput.addEventListener("change", function () {
    if (this.files.length > 0) {
      fileName.textContent = this.files[0].name;
    } else {
```

```
            fileName.textContent = "No file selected...";
        }
    });
  </script>

</body>
</html>
```

the upload functionality was tested by creating a sample text file and selecting it through the web interface. the file was then submitted and successfully stored in the `uploads` directory. the uploaded file was checked from the terminal to confirm that the file was created successfully and that the upload process was functioning correctly.



```
┌──(kali㉿kali)-[~]
└─$ cd ~/final-project/templates

┌──(kali㉿kali)-[~/final-project/templates]
└─$ cp upload.html upload.html.backup
cp command.html command.html.backup

┌──(kali㉿kali)-[~/final-project/templates]
└─$ ls -lh ~/final-project/templates
total 60K
-rw-rw-r-- 1 kali kali 4.9K Sep 11 04:03 command.html
-rw-rw-r-- 1 kali kali 4.9K Sep 11 08:08 command.html.backup
-rw------- 1 kali kali    0 Sep 11 08:08 command.html.save
-rw-rw-r-- 1 kali kali 8.6K Sep 11 07:09 index.html
-rw-rw-r-- 1 kali kali  14K Sep 11 08:01 upload.html
-rw-rw-r-- 1 kali kali  14K Sep 11 08:08 upload.html.backup

┌──(kali㉿kali)-[~/final-project/templates]
└─$ nano ~/final-project/templates/command.html

┌──(kali㉿kali)-[~/final-project/templates]
└─$ nano ~/final-project/templates/index.html

┌──(kali㉿kali)-[~/final-project/templates]
└─$ nano ~/final-project/app.py

┌──(kali㉿kali)-[~/final-project/templates]
└─$ nano ~/final-project/templates/upload.html

┌──(kali㉿kali)-[~/final-project/templates]
└─$
```

the file upload backend was implemented in app.py to receive the uploaded file and save it inside the configured uploads directory. the implementation obtains the submitted file from the request and constructs the destination path using the configured upload directory and the filename supplied by the user. the file is then saved directly to the destination without applying file type restrictions or filename sanitization.

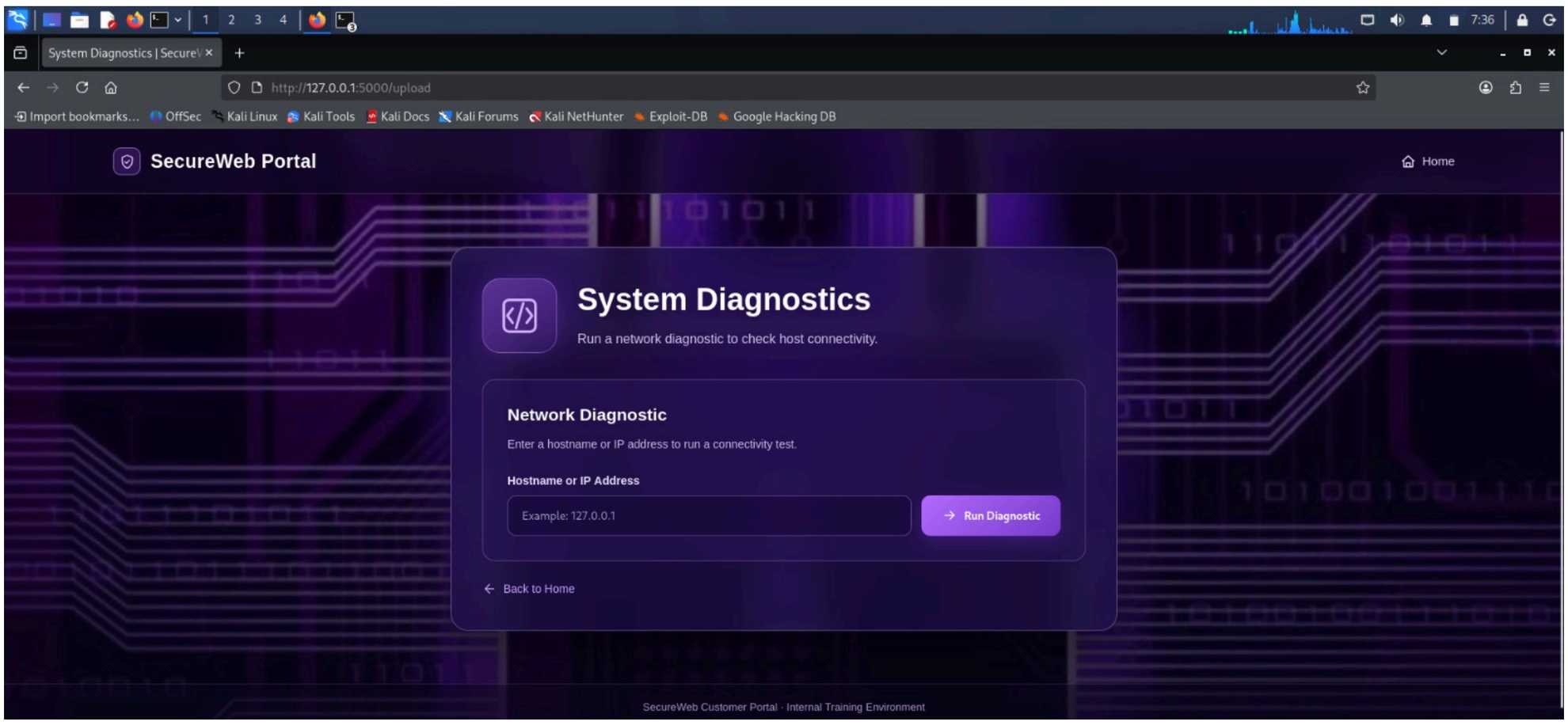
2.3.3 vulnerability 2 os command injection:

a system diagnostics functionality was implemented in the vulnerable web application using flask. the application provides a system diagnostics page where users can enter a hostname or ip address through the `/command` endpoint. the submitted value is used to construct a system `ping` command that is executed by the application.

system diagnostics page implementation:

the system diagnostics page was created using html and provides a network diagnostic interface. the page allows the user to enter a hostname or ip address and

submit the value to the application. the interface includes a hostname or ip address input field, an example value, and a button used to run the diagnostic.



the system diagnostics form sends the submitted hostname or ip address to the `/command` endpoint using a post request. the value entered by the user is received by the flask backend and used as part of the network diagnostic command.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>System Diagnostics | SecureWeb Portal</title>

  <style>
    *{
      box-sizing: border-box;
  }

  body{
    margin: 0;
    min-height: 100vh;
    font-family: Arial, sans-serif;
    color: white;

    background:
      linear-gradient(
        rgba(20, 5, 45, 0.55),
        rgba(8, 5, 25, 0.82)
```

```
        ),
        url("/static/upload-background.png");

    background-size: cover;
    background-position: center;
    background-repeat: no-repeat;
    background-attachment: fixed;

    display: flex;
    flex-direction: column;
}

.navbar{
    background: rgba(18, 5, 42, 0.72);
    backdrop-filter: blur(14px);
    -webkit-backdrop-filter: blur(14px);

    padding: 22px 7%;

    display: flex;
    justify-content: space-between;
    align-items: center;

    border-bottom: 1px solid rgba(255, 255, 255, 0.18);
}

.brand{
    display: flex;
    align-items: center;
    gap: 12px;

    font-size: 23px;
    font-weight: bold;
    letter-spacing: 0.5px;
}

.brand-icon{
    width: 34px;
    height: 34px;

    display: flex;
    align-items: center;
    justify-content: center;

    border: 1px solid rgba(210, 160, 255, 0.6);
```

```
    border-radius: 10px;

    background: rgba(166, 88, 255, 0.2);
  }

  .brand-icon svg {
    width: 21px;
    height: 21px;

    fill: none;
    stroke: #d9b7ff;
    stroke-width: 1.8;
    stroke-linecap: round;
    stroke-linejoin: round;
  }

  .nav-link {
    display: flex;
    align-items: center;
    gap: 8px;

    color: #eee2ff;
    text-decoration: none;
    font-size: 15px;

    transition: color 0.3s ease;
  }

  .nav-link:hover {
    color: white;
  }

  .nav-link svg {
    width: 19px;
    height: 19px;

    fill: none;
    stroke: currentColor;
    stroke-width: 1.8;
    stroke-linecap: round;
    stroke-linejoin: round;
  }

  .main-content {
    flex: 1;
```

```
    display: flex;
    align-items: center;
    justify-content: center;

    padding: 65px 20px;
}

.diagnostic-card{
    width: 100%;
    max-width: 820px;

    padding: 38px;

    background: rgba(31, 17, 75, 0.68);
    backdrop-filter: blur(20px);
    -webkit-backdrop-filter: blur(20px);

    border: 1px solid rgba(220, 184, 255, 0.35);
    border-radius: 22px;

    box-shadow:
      0 20px 65px rgba(0, 0, 0, 0.42),
      inset 0 1px 0 rgba(255, 255, 255, 0.12);
}

.heading-area {
    display: flex;
    align-items: center;
    gap: 25px;

    margin-bottom: 32px;
}

.main-icon{
    width: 92px;
    height: 92px;

    flex-shrink: 0;

    display: flex;
    align-items: center;
    justify-content: center;

    border-radius: 22px;
```

```
    background:
      linear-gradient(
        145deg,
        rgba(183, 111, 255, 0.4),
        rgba(90, 36, 170, 0.25)
      );

    border: 1px solid rgba(220, 184, 255, 0.35);

    box-shadow:
      0 0 30px rgba(171, 82, 255, 0.25);
  }

  .main-icon svg {
    width: 52px;
    height: 52px;

    fill: none;
    stroke: #d8b2ff;
    stroke-width: 1.5;
    stroke-linecap: round;
    stroke-linejoin: round;
  }

  .heading-text h1 {
    margin: 0 0 12px;

    font-size: 38px;
    color: white;

    text-shadow:
      0 4px 18px rgba(0, 0, 0, 0.35);
  }

  .heading-text p {
    margin: 0;

    color: #d8c8f2;
    font-size: 16px;
    line-height: 1.7;
  }

  .diagnostic-zone {
    padding: 30px;
```

```
    border: 1px solid rgba(207, 173, 255, 0.3);
    border-radius: 16px;

    background: rgba(42, 20, 91,0.38);
}

.diagnostic-zone h2{
    margin: 0 0 12px;

    color: white;
    font-size: 21px;
}

.diagnostic-zone p{
    margin: 0 0 25px;

    color: #d1c2eb;
    font-size: 14px;
    line-height: 1.7;
}

.form-label{
    display: block;

    margin-bottom: 10px;

    color: #eee4ff;
    font-size: 14px;
    font-weight: bold;
}

.form-row{
    display: flex;
    align-items: stretch;
    gap: 12px;
}

.host-input{
    flex: 1;
    min-width: 0;

    padding: 15px 17px;

    border: 1px solid rgba(207, 173, 255, 0.35);
```

```
    border-radius: 11px;

    background: rgba(20, 8, 55, 0.55);

    color: white;
    font-size: 14px;

    outline: none;

    transition:
      border-color 0.3s ease,
      background 0.3s ease;
}

.host-input::placeholder{
    color: #a99bc2;
}

.host-input:focus{
    border-color: #c89aff;
    background: rgba(31, 12, 76,0.7);
}

.run-button{
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: 9px;

    padding: 15px 25px;

    border: none;
    border-radius: 11px;

    background: linear-gradient(
      135deg,
      #b46cff,
      #783dcc
    );

    color: white;
    font-size: 14px;
    font-weight: bold;

    cursor: pointer;
```

```
    box-shadow:
      0 8px 22px rgba(104, 43, 185, 0.4);

    transition:
      transform 0.3s ease,
      box-shadow 0.3s ease;
}

.run-button:hover{
    transform: translateY(-2px);

    box-shadow:
      0 12px 28px rgba(104, 43, 185, 0.58);
}

.run-button svg{
    width: 19px;
    height: 19px;

    fill: none;
    stroke: currentColor;
    stroke-width: 1.8;
    stroke-linecap: round;
    stroke-linejoin: round;
}

.result-box{
    margin-top: 25px;
}

.result-title{
    margin-bottom: 10px;

    color: #eee4ff;
    font-size: 14px;
    font-weight: bold;
}

.result{
    min-height: 80px;
    max-height: 300px;

    overflow: auto;
```

```
    margin: 0;
    padding: 18px;

    border: 1px solid rgba(207, 173, 255, 0.3);
    border-radius: 11px;

    background: rgba(10, 4, 30, 0.65);

    color: #d9c5f8;
    font-family: "Courier New", monospace;
    font-size: 13px;
    line-height: 1.6;

    white-space: pre-wrap;
}

.back-link{
    display: inline-flex;
    align-items: center;
    gap: 8px;

    margin-top: 25px;

    color: #cda4ff;
    text-decoration: none;
    font-size: 14px;

    transition: color 0.3s ease;
}

.back-link:hover{
    color: white;
}

.back-link svg{
    width: 18px;
    height: 18px;

    fill: none;
    stroke: currentColor;
    stroke-width: 1.8;
    stroke-linecap: round;
    stroke-linejoin: round;
}
```

```css
.footer{
    padding: 20px;

    text-align: center;

    color: #a99ac7;
    font-size: 13px;

    border-top: 1px solid rgba(255, 255, 255, 0.12);
    background: rgba(10, 4, 28, 0.45);
}

@media (max-width: 650px){
    .navbar{
        padding: 20px;
    }

    .brand{
        font-size: 18px;
    }

    .main-content{
        padding: 35px 15px;
    }

    .diagnostic-card{
        padding: 25px 20px;
    }

    .heading-area{
        align-items: flex-start;
        gap: 16px;
    }

    .main-icon{
        width: 65px;
        height: 65px;
        border-radius: 17px;
    }

    .main-icon svg{
        width: 38px;
        height: 38px;
    }
```

```
    .heading-text h1{
      font-size: 28px;
    }

    .heading-text p{
      font-size: 14px;
    }

    .form-row{
      flex-direction: column;
    }

    .run-button{
      width: 100%;
    }
  }
  </style>
</head>

<body>

  <nav class="navbar">

    <div class="brand">
      <div class="brand-icon">
        <svg viewBox="0 0 24 24" aria-hidden="true">
          <path d="M12 3l7 3v5c0 4.5-3 8.2-7 10-4-1.8-7-5.5-7-10V6l7-3z"></path>
          <path d="M9 12l2 2 4-4"></path>
        </svg>
      </div>

      <span>SecureWeb Portal</span>
    </div>

    <a class="nav-link" href="/">
      <svg viewBox="0 0 24 24" aria-hidden="true">
        <path d="M3 11l9-8 9 8"></path>
        <path d="M5 10v10h14V10"></path>
        <path d="M9 20v-6h6v6"></path>
      </svg>

      Home
    </a>

  </nav>
```

```
<main class="main-content">

  <section class="diagnostic-card">

    <div class="heading-area">

      <div class="main-icon">
        <svg viewBox="0 0 24 24" aria-hidden="true">
          <rect x="3" y="3" width="18" height="18" rx="3"></rect>
          <path d="M8 8l-3 4 3 4"></path>
          <path d="M16 8l3 4-3 4"></path>
          <path d="M14 5l-4 14"></path>
        </svg>
      </div>

      <div class="heading-text">
        <h1>System Diagnostics</h1>

        <p>
          Run a network diagnostic to check
          host connectivity.
        </p>
      </div>

    </div>

  <div class="diagnostic-zone">

    <h2>Network Diagnostic</h2>

    <p>
      Enter a hostname or IP address to run a
      connectivity test.
    </p>

    <form method="POST">

      <label class="form-label" for="host">
        Hostname or IP Address
      </label>

      <div class="form-row">

        <input
```

```
            class="host-input"
            type="text"
            id="host"
            name="host"
            placeholder="Example: 127.0.0.1"
            required
          >

          <button class="run-button" type="submit">

            <svg viewBox="0 0 24 24" aria-hidden="true">
              <path d="M5 12h14"></path>
              <path d="M13 6l6 6-6 6"></path>
            </svg>

            Run Diagnostic

          </button>

        </div>

      </form>

      {% if result %}
      <div class="result-box">

        <div class="result-title">
          Diagnostic Result
        </div>

        <pre class="result">{{ result }}</pre>

      </div>
      {% endif %}

    </div>

    <a class="back-link" href="/">
      <svg viewBox="0 0 24 24" aria-hidden="true">
        <path d="M19 12H5"></path>
        <path d="M12 19l-7-7 7-7"></path>
      </svg>

      Back to Home
    </a>
```

```
    </section>

  </main>

  <footer class="footer">
    SecureWeb Customer Portal · Internal Training Environment
  </footer>

</body>
</html>
```

the flask backend was then configured to process the submitted host value and execute a ping command. the application stores the command output in a result variable and returns it to the system diagnostics page so that the output can be displayed to the user

```
from flask import Flask, request, render_template, redirect, url_for

import os
import subprocess

app = Flask(__name__)

UPLOAD_FOLDER = "uploads"

app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER

os.makedirs(UPLOAD_FOLDER, exist_ok=True)


@app.route("/")
def index():
    return render_template("index.html")


@app.route("/upload", methods=["GET", "POST"])
def upload_file():

    if request.method == "POST":

        file = request.files["file"]

        if file:
```

```
        filepath = os.path.join(
            app.config["UPLOAD_FOLDER"],
            file.filename
        )

        file.save(filepath)

        return redirect(url_for("upload_file"))

    return render_template("upload.html")


@app.route("/command", methods=["GET", "POST"])
def command():

    result = ""

    if request.method == "POST":

        host = request.form["host"]

        command = f"ping -c 1 {host}"

        result = subprocess.getoutput(command)

    return render_template("command.html", result=result)


if __name__ == "__main__":
    app.run(
        host="0.0.0.0",
        port=5000,
        debug=False
    )
```

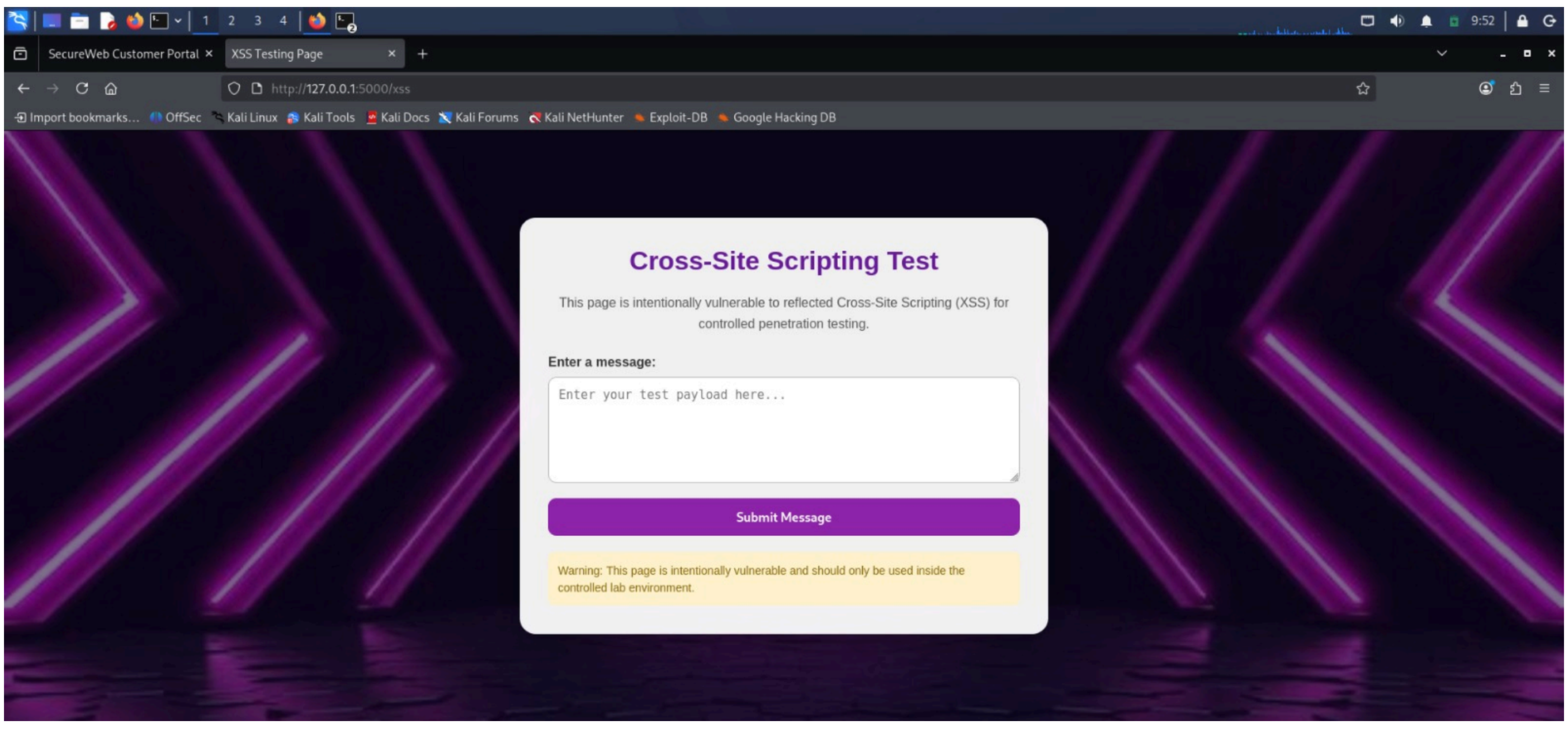
the command is constructed using the user-supplied host value and executed through `subprocess.getoutput()`. because the input is directly inserted into a shell command without validation or sanitization, the functionality is vulnerable to os command injection. this behavior was intentionally implemented as part of the vulnerable application so that the command injection vulnerability can be assessed during the penetration testing phase.

2.3.4 vulnerability 3 cross-site scripting (xss):

a reflected cross-site scripting (xss) vulnerability was implemented in the vulnerable web application through the `/xss` route. this functionality allows the user to submit a message

through a web form. the submitted input is then displayed directly on the same page without proper input sanitization or output encoding.

xss page implementation:



the xss page was designed using html and css to provide a simple message submission form. the page contains a text input field where the user can enter a message and a submit button to send the input to the flask application. the submitted message is displayed below the form after the request is processed.

the vulnerability was introduced by rendering the submitted message using the `safe` filter. this filter prevents flask and jinja2 from escaping html characters, allowing any html or javascript code included in the input to be interpreted by the browser.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>XSS Testing Page</title>

  <style>
    * {
      box-sizing: border-box;
```

```
}

body{
    margin: 0;
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    font-family: Arial, sans-serif;

    background-image:
        linear-gradient(
            rgba(20, 10, 45, 0.55),
            rgba(20, 10, 45, 0.55)
        ),
        url('/static/xss-test.png.jpg');

    background-size: cover;
    background-position: center;
    background-repeat: no-repeat;
}

.container{
    width: 90%;
    max-width: 650px;
    padding: 35px;
    border-radius: 20px;

    background: rgba(255, 255, 255, 0.94);
    box-shadow: 0 15px 40px rgba(0, 0, 0, 0.35);
}

h1{
    margin-top: 0;
    text-align: center;
    color: #6a1b9a;
}

.description{
    margin-bottom: 25px;
    text-align: center;
    color: #555;
    line-height: 1.6;
}
```

```css
label{
    display: block;
    margin-bottom: 8px;
    font-weight: bold;
    color: #333;
}

textarea{
    width: 100%;
    min-height: 130px;
    padding: 12px;
    border: 1px solid #bbb;
    border-radius: 10px;
    resize: vertical;
    font-size: 15px;
}

textarea:focus{
    outline: none;
    border-color: #8e24aa;
    box-shadow: 0 0 5px rgba(142, 36, 170, 0.35);
}

button{
    width: 100%;
    margin-top: 18px;
    padding: 13px;
    border: none;
    border-radius: 10px;

    background: #8e24aa;
    color: white;
    font-size: 16px;
    font-weight: bold;
    cursor: pointer;
}

button:hover{
    background: #6a1b9a;
}

.result{
    margin-top: 25px;
    padding: 18px;
    border-radius: 10px;
```

```
      background: #f3e5f5;
      border-left: 5px solid #8e24aa;
      color: #333;
      word-wrap: break-word;
    }

    .warning{
      margin-top: 20px;
      padding: 12px;
      border-radius: 8px;
      background: #fff3cd;
      color: #856404;
      font-size: 14px;
      line-height: 1.5;
    }
  </style>
</head>

<body>

  <div class="container">
    <h1>Cross-Site Scripting Test</h1>

    <p class="description">
      This page is intentionally vulnerable to reflected Cross-Site
      Scripting (XSS) for controlled penetration testing.
    </p>

    <form method="POST" action="/xss">
      <label for="message">Enter a message:</label>

      <textarea
        id="message"
        name="message"
        placeholder="Enter your test payload here..."
        required
      ></textarea>

      <button type="submit">Submit Message</button>
    </form>

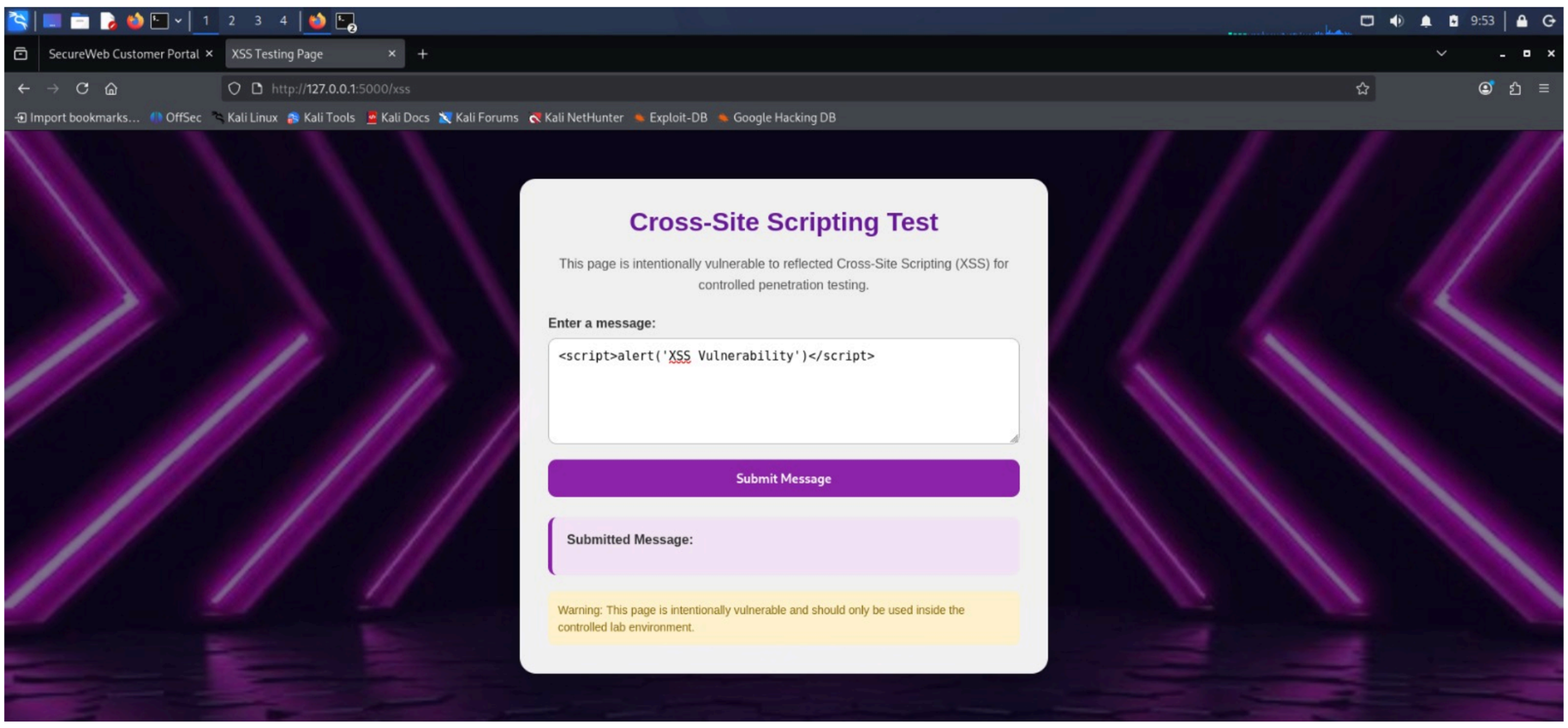
    {% if message %}
    <div class="result">
      <strong>Submitted Message:</strong>
      <p>{{ message | safe }}</p>
```

```
    </div>
    {% endif %}

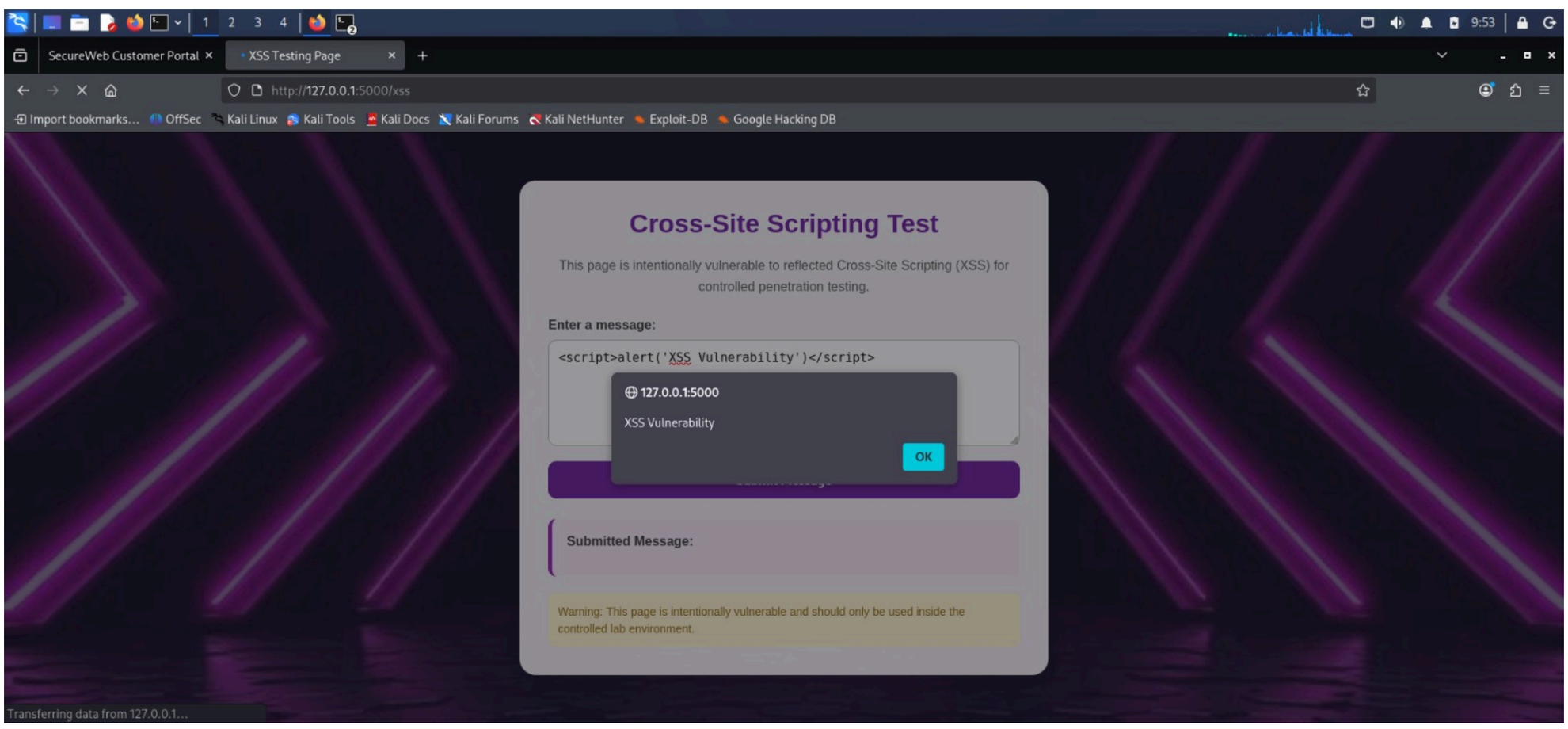
    <div class="warning">
      Warning: This page is intentionally vulnerable and should only
      be used inside the controlled lab environment.
    </div>
  </div>

</body>
</html>
```

xss vulnerability testing:



the vulnerability was tested by submitting a javascript payload through the message input field. the payload used for testing was:



after submitting the payload, the browser executed the javascript code and displayed an alert message containing “xss vulnerability”. this confirms that the application is vulnerable to reflected cross-site scripting because user-controlled input is rendered as executable javascript instead of being safely encoded.

security impact:

the xss vulnerability allows an attacker to inject and execute arbitrary javascript code in the browser of a user who accesses the affected page. this could allow attackers to display fake login forms, modify page content, steal sensitive information stored in the browser, perform actions on behalf of the victim, or redirect the user to malicious websites.

3.2.1 unrestricted file upload:

the file upload functionality of the vulnerable web application was examined to determine whether the application restricts the types of files that can be uploaded. the upload page was first inspected to identify the request method and the name of the upload parameter. the form was found to use an http post request with a multipart file upload field. a normal text file was uploaded first to verify that the upload functionality was working correctly. the request was accepted by the application and returned a successful redirect response. the uploaded file was then located inside the application's upload directory on the server filesystem.

after confirming that normal file uploads were accepted, additional files with different extensions were submitted. a php file and a python file were uploaded successfully using the same upload functionality. both files were stored in the upload

directory alongside the original text file. this confirmed that the application does not enforce an extension allowlist or restrict uploads to a specific set of safe file types. application also did not appear to perform sufficient validation of the uploaded file content. the upload process was based on accepting the submitted filename and storing the file, without demonstrating file type validation, content inspection, or restrictions on potentially executable extensions.

an additional request was made to access the uploaded file through the web application. the request returned a not found response because the application did not provide a route for directly serving files from the upload directory. therefore, the testing confirmed unrestricted file storage, but it did not demonstrate that the uploaded files could be executed remotely.

this vulnerability may allow attackers to upload malicious files, server-side scripts, or files containing harmful content. if the upload directory is later exposed through the web server or configured to execute scripts, the vulnerability could lead to further attacks, including remote code execution. uploaded files may also be used to consume server storage or distribute malicious content to other users. issue was classified as an unrestricted file upload vulnerability. remote code execution was not claimed because no successful execution of an uploaded file was demonstrated during the test.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV -sC 127.0.0.1 -p 5000
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-11 10:27 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0015s latency).

PORT     STATE SERVICE VERSION
5000/tcp open  http    Werkzeug httpd 3.1.8 (Python 3.14.6)
|_http-title: SecureWeb Customer Portal
|_http-server-header: Werkzeug/3.1.8 Python/3.14.6

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.44 seconds

┌──(kali㉿kali)-[~]
└─$ mkdir -p ~/pentest-evidence
nmap -sV -sC 127.0.0.1 -p 5000 -oN ~/pentest-evidence/nmap-scan.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-11 10:28 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00011s latency).

PORT     STATE SERVICE VERSION
5000/tcp open  http    Werkzeug httpd 3.1.8 (Python 3.14.6)
|_http-title: SecureWeb Customer Portal
|_http-server-header: Werkzeug/3.1.8 Python/3.14.6

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.33 seconds

┌──(kali㉿kali)-[~]
└─$
```

```
┌──(kali㉿kali)-[~]
└─$ echo "test upload" > ~/pentest-evidence/test.txt

┌──(kali㉿kali)-[~]
└─$ curl -i -X POST -F "file=@/home/kali/pentest-evidence/test.txt" http://127.0.0.1:5000/upload
HTTP/1.1 302 FOUND
Server: Werkzeug/3.1.8 Python/3.14.6
Date: Fri, 11 Sep 2026 14:31:33 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 201
Location: /upload
Connection: close

<!doctype html>
<html lang=en>
<title>Redirecting ... </title>
<h1>Redirecting ... </h1>
<p>You should be redirected automatically to the target URL: <a href="/upload">/upload</a>. If not, click the link.

┌──(kali㉿kali)-[~]
└─$
```

```
└─$ find ~/final-project -type f -name "test.txt" -o -name "test.py"
/home/kali/final-project/test.txt
/home/kali/final-project/uploads/test.txt
/home/kali/final-project/venv/lib/python3.14/site-packages/werkzeug/test.py

┌──(kali㉿kali)-[~]
└─$ grep -RniE "upload|save|uploads|secure_filename|send_from_directory" ~/final-project --exclude-dir=venv
/home/kali/final-project/app.py:8:UPLOAD_FOLDER = "uploads"
/home/kali/final-project/app.py:10:app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER
/home/kali/final-project/app.py:12:os.makedirs(UPLOAD_FOLDER, exist_ok=True)
/home/kali/final-project/app.py:20:@app.route("/upload", methods=["GET", "POST"])
/home/kali/final-project/app.py:21:def upload_file():
/home/kali/final-project/app.py:30:                app.config["UPLOAD_FOLDER"],
/home/kali/final-project/app.py:34:            file.save(filepath)
/home/kali/final-project/app.py:36:            return redirect(url_for("upload_file"))
/home/kali/final-project/app.py:38:    return render_template("upload.html")
/home/kali/final-project/test.txt:1:This is a test upload file
grep: /home/kali/final-project/__pycache__/app.cpython-314.pyc: binary file matches
/home/kali/final-project/templates/command.html:24:                url("/static/upload-background.png");
/home/kali/final-project/templates/upload.html:6:    <title>File Upload | SecureWeb Portal</title>
/home/kali/final-project/templates/upload.html:24:                url("/static/upload-background.png");
/home/kali/final-project/templates/upload.html:121:        .upload-card {
/home/kali/final-project/templates/upload.html:201:        .upload-zone {
/home/kali/final-project/templates/upload.html:223:        .upload-zone:hover {
/home/kali/final-project/templates/upload.html:246:        .upload-zone h2 {
/home/kali/final-project/templates/upload.html:253:        .upload-zone p {
/home/kali/final-project/templates/upload.html:317:        .upload-button {
/home/kali/final-project/templates/upload.html:348:        .upload-button:hover {
/home/kali/final-project/templates/upload.html:355:        .upload-button svg {
/home/kali/final-project/templates/upload.html:420:            .upload-card {
/home/kali/final-project/templates/upload.html:453:            .upload-button {
/home/kali/final-project/templates/upload.html:489:        <section class="upload-card">
/home/kali/final-project/templates/upload.html:503:                <h1>File Upload</h1>
/home/kali/final-project/templates/upload.html:506:                    Upload files through the customer portal.
/home/kali/final-project/templates/upload.html:515:                <div class="upload-zone">
/home/kali/final-project/templates/upload.html:550:                    <button class="upload-button" type="submit">
/home/kali/final-project/templates/upload.html:559:                        Upload
/home/kali/final-project/templates/index.html:309:            <!-- File Upload Card -->
/home/kali/final-project/templates/index.html:321:                <h2>File Upload</h2>
/home/kali/final-project/templates/index.html:324:                    Upload files through the customer portal.
/home/kali/final-project/templates/index.html:328:                <a class="button" href="/upload">
/home/kali/final-project/templates/index.html:329:                    Open File Upload
/home/kali/final-project/templates/upload.html.backup:6:    <title>File Upload | SecureWeb Portal</title>
/home/kali/final-project/templates/upload.html.backup:24:                url("/static/upload-background.png");
/home/kali/final-project/templates/upload.html.backup:121:        .upload-card {
/home/kali/final-project/templates/upload.html.backup:201:        .upload-zone {
/home/kali/final-project/templates/upload.html.backup:223:        .upload-zone:hover {
/home/kali/final-project/templates/upload.html.backup:246:        .upload-zone h2 {
```

```
┌──(kali㉿kali)-[~]
└─$ echo 'print("uploaded python file")' > ~/pentest-evidence/test.py

┌──(kali㉿kali)-[~]
└─$ curl -i -X POST -F "file=@/home/kali/pentest-evidence/test.py" http://127.0.0.1:5000/upload
HTTP/1.1 302 FOUND
Server: Werkzeug/3.1.8 Python/3.14.6
Date: Fri, 11 Sep 2026 14:35:10 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 201
Location: /upload
Connection: close

<!doctype html>
<html lang=en>
<title>Redirecting ... </title>
<h1>Redirecting ... </h1>
<p>You should be redirected automatically to the target URL: <a href="/upload">/upload</a>. If not, click the link.

┌──(kali㉿kali)-[~]
└─$ find ~/final-project/uploads -maxdepth 1 -type f -printf '%f\n'
test.txt
test.php
test.py

┌──(kali㉿kali)-[~]
```

3.2.2 os command injection:

the network diagnostic functionality was tested to determine whether user-supplied host values were safely handledbefore being passed to the operating system. the form accepts a hostname or ip address through the host parameter and normally executes a ping command against the supplied value.

a valid host value was first combined with an additional command using a semicolon separator. the submitted value was 127.0.0.1; id. the application returned the normal ping output followed by the result of the id command, including the current user, group, and group memberships. this confirmed that the additional command was executed by the operating system. a second test used the whoami command in the same way. the response returned kali, confirming that commands supplied after the original host value were executed with the privileges of the application process.

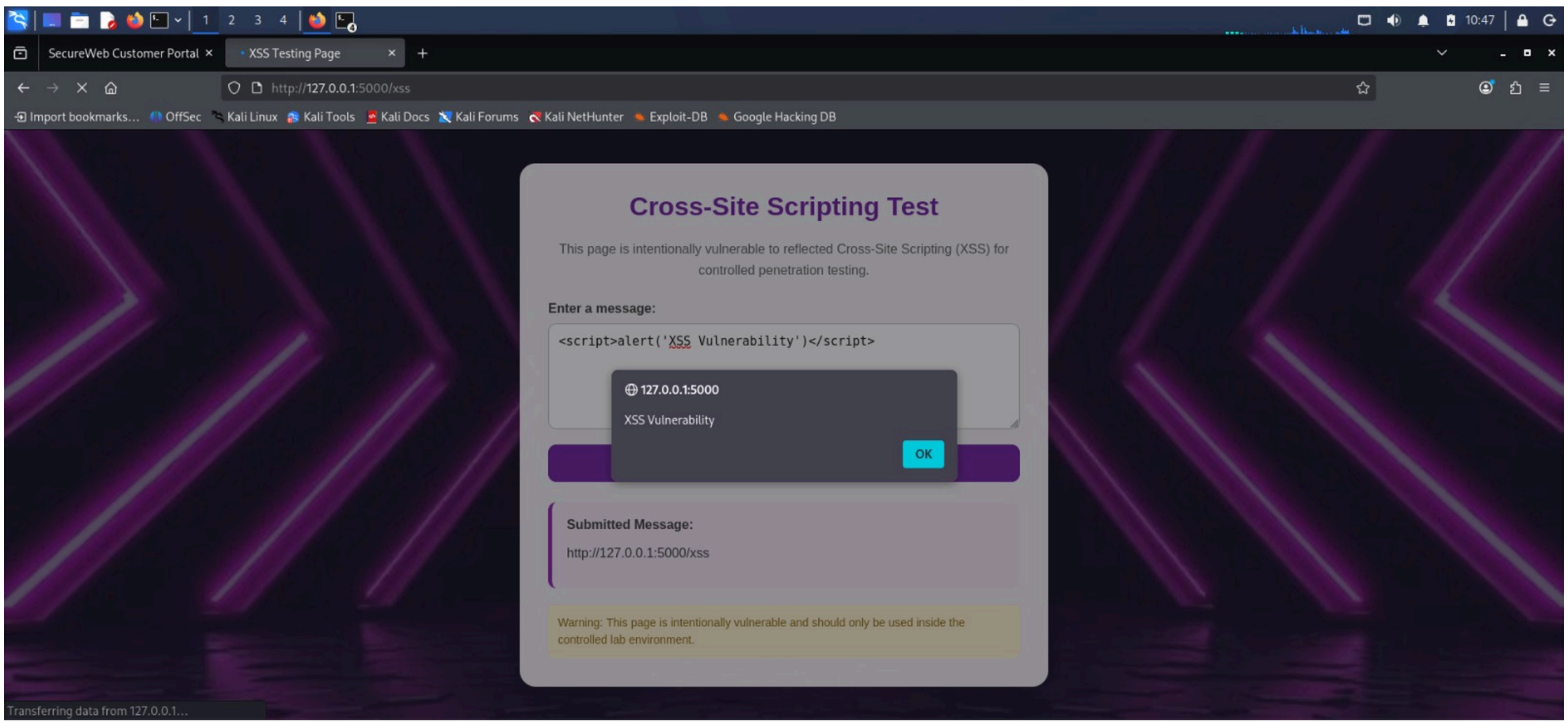
the issue occurs because user input is passed into an operating system command without proper validation or safe argument handling. an attacker could use this weakness to execute arbitrary commands, read sensitive files, modify application data, collect system information, or perform further attacks depending on the privileges of the running process. vulnerability was classified as os command injection. the successful execution of both id and whoami provides direct evidence that the application is vulnerable to command injection.



the response returned the submitted payload directly inside the html response. the screenshot shows that the submitted javascript payload was reflected in the response without html encoding.

this confirmed that the application reflected the user-controlled input into the html response without encoding the html characters. the presence of the `<script>` element in the returned response demonstrated that the input was not safely escaped before being rendered.

```
┌──(kali㉿kali)-[~]
└─$ curl -s -X POST --data-urlencode "message=<script>alert('XSS Vulnerability')</script>" http://127.0.0.1:5000/xss | grep -nE "script|XSS|alert"
7:    <title>XSS Testing Page</title>
50:         .description {
125:        <p class="description">
127:            Scripting (XSS) for controlled penetration testing.
146:            <p><script>alert('XSS Vulnerability')</script></p>
```



4. incident response and detection:

4.1 incident investigation:

the incident investigation focused on identifying and analyzing exploitation attempts against the vulnerable flask web application. the investigation used the application logs, bash history, uploaded file artifacts, and filesystem checks to reconstruct the activity performed against the application.

the investigation confirmed three relevant attack activities: os command injection through the `/command` endpoint, reflected cross-site scripting through the `/xss` endpoint, and unrestricted file upload through the `/upload` endpoint. the file upload functionality was confirmed to accept and store uploaded files, but no direct upload-to-rce execution was demonstrated.

4.2 application log analysis:

the flask application was configured to write its runtime output to `flask.log`. the application logs recorded incoming http requests and their response status codes. analysis of the log showed successful requests to the vulnerable endpoints used during testing.

the command injection request generated a `200` response, the xss request generated a `200` response, and the file upload request generated a `302` redirect after the file was successfully processed.



4.3 bash history analysis :

bash history was reviewed to identify the commands used during the exploitation and validation process. the history contained the command injection payloads using `id` and `whoami`, as well as the reflected xss payload containing a `<script>` element and an `alert()` function.

the command injection testing demonstrated that user-controlled input was passed to the operating system command execution mechanism. the xss testing demonstrated that html and javascript content supplied by the user was reflected into the application response without appropriate output encoding.





4.4 correlated attack timeline:

the available evidence was correlated using timestamps from the flask application log and the corresponding testing activity. the resulting timeline shows the sequence of exploitation attempts against the vulnerable application.

time	activity	endpoint	result
11:26:44	os command injection attempt	`/command`	http 200
11:26:51	reflected xss attempt	`/xss`	http 200

time	activity	endpoint	result
11:26:58	file upload attempt	`/upload`	http 302

the timeline demonstrates that the three vulnerable functions were tested sequentially within the same application session.

```
┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ echo "== xss ==" && grep -nE 'POST /xss HTTP' flask.log
== xss ==
9:127.0.0.1 - - [11/Sep/2026 11:26:51] "POST /xss HTTP/1.1" 200 -
```

```
┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ echo "== file upload ==" && grep -nE 'POST /upload HTTP' flask.log
== file upload ==
10:127.0.0.1 - - [11/Sep/2026 11:26:58] "POST /upload HTTP/1.1" 302 -
```

```
┌──(venv)─(kali㉿kali)-[~/final-project]
└─$ echo "== uploaded artifacts ==" && find uploads -type f -newermt "2026-09-11 11:20:00" -printf '%TY-%Tm-%Td %TH:%TM:%TS %p\n'
== uploaded artifacts ==
2026-09-11 11:26:58.0898580840 uploads/test.txt
```

4.5 containment and artifact analysis:

the application was running as a local flask development service on port 5000. the investigation remained limited to the local lab environment. no evid

5. vulnerability remediation and secure coding:

5.1 remediation overview:

following the vulnerability assessment and exploitation phases, the identified vulnerabilities in the vulnerable flask web application were remediated using secure coding practices. the remediation focused on three vulnerabilities: insecure file upload, operating system command injection, and reflected cross-site scripting (xss).

each vulnerability was addressed separately and tested after applying the security controls. a final verification was then performed to confirm that the original attack techniques were no longer successful. git was also used to maintain a clear and traceable history of the remediation changes.

5.2 vulnerability 1: insecure file upload remediation:

the original file upload functionality accepted uploaded files without restricting their file extensions. this allowed unsupported file types, such as php files, to be uploaded to the application.

to mitigate this vulnerability, the upload functionality was modified to validate the uploaded filename and extension. `secure_filename()` was used to sanitize filenames, while an allowlist was implemented to permit only `txt`, `png`, `jpg`, and `jpeg` files.

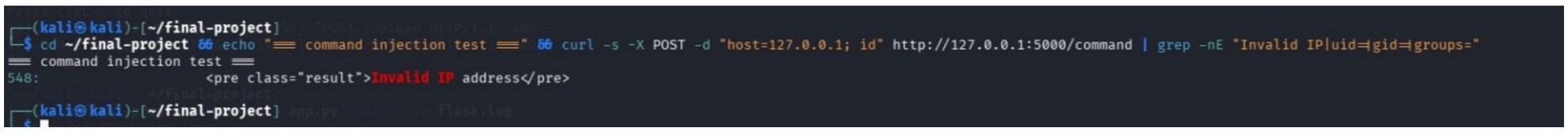
the remediated functionality was tested using both a malicious php file and a legitimate text file. the malicious `test.php` file was rejected with an http `400` response, while the legitimate `test.txt` file was accepted and returned an http `302` response.



these results demonstrate that unsupported file types are no longer accepted while legitimate uploads continue to function correctly.

5.3 vulnerability 2: command injection remediation:

the original command functionality was vulnerable to command injection because user-controlled input was directly incorporated into a shell command. an attacker could append shell commands to the expected host value. vulnerability was remediated by validating the supplied host value using python's `ipaddress` module. only valid ip addresses are accepted. additionally, the ping operation was changed to use `subprocess.run()` with a list of arguments instead of constructing a shell command from user input. previous command injection payload `127.0.0.1; id` was tested again after the remediation. the application rejected the input and returned `Invalid IP address`.



the result confirms that the injected `id` command was not executed and that the malicious input was rejected as an invalid ip address.

5.4 vulnerability 3: reflected cross-site scripting (xss) remediation:

the original `/xss` functionality displayed user-supplied input using the jinja `safe` filter. this disabled the framework's normal html escaping and allowed attacker-controlled html and javascript to be interpreted by the browser.vulnerability was remediated by removing the `safe` filter and allowing jinja's default output escaping to process the submitted input.original xss payload `<script>alert('XSS Vulnerability')</script>` was submitted again after the remediation. the application returned the payload as escaped html rather than executable javascript.



the escaped output confirms that the `<script>` element was treated as text and was not executed by the application.

5.5 git version control and remediation tracking:

git was used throughout the remediation process to maintain a traceable history of the security changes. a vulnerable baseline was created first, followed by individual commits for the file upload, command injection, and xss remediations.

final git history showed three remediation commits:



this provides a clear record of the implemented security improvements and allows each vulnerability remediation to be traced to a specific change.

5.6 final remediation verification:

after implementing all three security fixes, a final verification was performed using the same vulnerability scenarios that were used during the initial testing. file upload test confirmed that the malicious php file was rejected while the legitimate text file was accepted. the command injection test confirmed that the malicious input was rejected as an invalid ip address. finally, the xss test confirmed that the malicious javascript payload was html-escaped.



the final verification demonstrated that all three previously identified vulnerabilities were successfully mitigated while the legitimate functionality of the application remained available.

5.7 remediation conclusion:

the remediation phase successfully addressed the three vulnerabilities identified during the security assessment. insecure file upload handling was mitigated through

filename sanitization and extension allowlisting. command injection was prevented through strict ip validation and safe subprocess execution. reflected xss was mitigated by removing the `safe` filter and applying jinja's default html escaping.final verification confirmed that the original attack inputs were no longer successfully executed. the use of git also provided a traceable history of the remediation process and the individual security fixes.overall, the application was successfully hardened against the identified vulnerabilities using practical secure coding controls while preserving its intended functionality.

Section 6 Conclusion:

the project demonstrated a complete cybersecurity lifecycle covering application deployment, vulnerability assessment, exploitation, incident investigation, detection, mitigation, and re-exploitation testing.

three vulnerabilities were identified and investigated in the vulnerable flask web application: insecure file upload, operating system command injection, and reflected cross-site scripting. the penetration testing phase demonstrated that these vulnerabilities could be triggered through attacker-controlled input and provided practical evidence of their security impact.

incident response and detection phase analyzed application activity and bash history to reconstruct the observed attack activity. detection queries were developed to identify file upload attempts, xss requests, and command injection activity. uploaded artifacts and potential persistence locations were also examined as part of the post-compromise investigation.

during the mitigation phase, secure coding controls were implemented for all identified vulnerabilities. file upload handling was secured through filename sanitization and extension allowlisting. command injection was mitigated through strict ip address validation and safe argument-based subprocess execution. reflected xss was mitigated by removing the unsafe jinja `safe` filter and restoring default html output escaping.

git version control was used to maintain a traceable remediation history, beginning with a vulnerable baseline and followed by separate commits for each security fix. the patched application was then tested using the same attack techniques used during the initial assessment. the malicious file upload and command injection payloads were rejected, while the xss payload was returned as escaped html rather than executable javascript.

overall, the project demonstrated that secure coding practices, systematic security testing, incident detection, and version-controlled remediation can be combined to reduce the attack surface of a vulnerable web application. the final verification

confirmed that the identified vulnerabilities were successfully mitigated while the application's legitimate functionality remained available.

References:

[1] OWASP Foundation, “OWASP Top 10: Web Application Security Risks.” https://owasp.org/www-project-top-ten/

[2] OWASP Foundation, “File Upload Cheat Sheet.” https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[3] OWASP Foundation, “OS Command Injection Defense Cheat Sheet.” https://cheatsheetseries.owasp.org/cheatsheets/OS_Command_Injection_Defense_Cheat_Sheet.html

[4] OWASP Foundation, “Cross Site Scripting Prevention Cheat Sheet.” https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[5] Pallets Projects, “Flask Documentation.” https://flask.palletsprojects.com/

[6] Pallets Projects, “Jinja Documentation.” https://jinja.palletsprojects.com/

[7] Pallets Projects, “Werkzeug Documentation.” https://werkzeug.palletsprojects.com/

[8] Python Software Foundation, “Python Documentation.” https://docs.python.org/3/

[9] Python Software Foundation, “ipaddress — IPv4/IPv6 manipulation library.” https://docs.python.org/3/library/ipaddress.html

[10] Python Software Foundation, “subprocess — Subprocess management.” https://docs.python.org/3/library/subprocess.html

[11] Git, “Git Documentation.” https://git-scm.com/doc

By : Danh khaled obaidat